

VoteChain: A Decentralised Voting System Using Ethereum Smart Contracts

Yash Yadav

Department of Physics, Indian Institute of Technology Kanpur

2024-25

Abstract. Traditional electronic voting systems suffer from single points of failure, opaque audit trails, and centralised control that requires trust in an authority. This project presents *VoteChain*, a decentralised voting platform built on the Ethereum blockchain. The system enforces election integrity through immutable smart-contract logic written in Solidity, role-based access control separating super-admin and voter roles, and event-driven data indexing that enables a gas-free read interface. The frontend, built with React and Ethers.js v6, provides a responsive dashboard for election management and a voter portal with MetaMask wallet authentication. The contract is deployed on the Ethereum Sepolia testnet. A commit-reveal voting scheme is prototyped to extend the system toward vote confidentiality.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Scope and Limitations	3
2	Background	3
2.1	Ethereum and Smart Contracts	3
2.2	Solidity	3
2.3	MetaMask and Ethers.js	3
3	System Architecture	4
3.1	Overview	4
3.2	Data Flow	4
3.3	Component Summary	4
4	Smart Contract Design	4
4.1	State Variables and Data Structures	4
4.2	Election Lifecycle	5
4.3	Access Control	5
4.4	Vote Integrity	5
4.5	Event Architecture	6
5	Frontend Implementation	6
5.1	Architecture Decisions	6
5.2	Key Frontend Features	7
5.3	Event Querying Strategy	7
5.4	Commit-Reveal Privacy Prototype	7
6	Security Analysis	8
6.1	Threats Addressed	8

6.2	Known Limitations	8
6.3	Gas Analysis	8
7	Deployment	9
7.1	Technology Stack	9
7.2	Deployment Process	9
8	Results and Demo Flow	9
9	Discussion	10
9.1	What Blockchain Solves Here	10
9.2	Comparison with EVM-based Electronic Voting	10
10	Future Work	10
11	Conclusion	10

1. Introduction

Elections are foundational to democratic systems, yet existing electronic voting infrastructure frequently concentrates control in a single authority, making comprehensive external audit difficult. Paper-based systems are slow and prone to physical manipulation. Blockchain-based voting addresses the integrity and auditability dimensions of this problem by recording votes as cryptographically signed on-chain transactions that are visible to all participants and modifiable by none.

1.1 Motivation

Recent public discourse around election integrity—including concerns about EVM tampering, ghost voters, and post-count manipulation raised in the context of Indian parliamentary elections—motivates a system in which:

- Every vote is an immutable on-chain transaction.
- Counting is deterministic and publicly verifiable.
- No central authority controls data post-submission.
- The complete audit trail is available to any observer.

1.2 Scope and Limitations

Blockchain guarantees *integrity* (votes cannot be modified) and *transparency* (totals are publicly verifiable). It does not, by itself, guarantee voter *identity* (a wallet is not a verified person) or vote *privacy* (public chains are fully transparent). These limitations are explicitly acknowledged in the design and addressed architecturally: identity verification is delegated to an off-chain admin whitelist, and a commit-reveal protocol is prototyped for confidentiality.

2. Background

2.1 Ethereum and Smart Contracts

Ethereum is a decentralised, Turing-complete blockchain on which programs called *smart contracts* execute deterministically across all network nodes. A smart contract deployed to an address is immutable; its logic cannot be changed after deployment. Transactions that call contract functions are cryptographically signed by the sender's private key, creating a tamper-evident audit trail. Gas fees denominated in Ether compensate miners for computation.

2.2 Solidity

Solidity is the primary high-level language for Ethereum smart contracts. It compiles to EVM bytecode. Key features used in this project include: *mappings* (hash-map storage), *enums* (election state machine), *events* (cheap on-chain logging queried by the frontend), and *modifiers* (access control predicates applied to functions).

2.3 MetaMask and Ethers.js

MetaMask is a browser extension that manages Ethereum key pairs and signs transactions on behalf of the user. Ethers.js v6 is a JavaScript library that provides a high-level API for connecting to Ethereum nodes, creating contract instances, and querying event logs without gas cost.

3. System Architecture

3.1 Overview

Architecture: Voter/Admin Browser (React + Ethers.js + MetaMask) → Ethereum RPC (Alchemy / public Sepolia) → Deployed `VotingSystem` contract (Sepolia)

All persistent state lives on-chain. The frontend is a pure read/write client with no backend server. Read operations (fetching election lists, candidate lists, voter lists, vote counts) query the contract's event logs — a gas-free operation using the Ethereum JSON-RPC `eth_getLogs` call. Write operations (creating elections, casting votes, etc.) send signed transactions via MetaMask.

3.2 Data Flow

1. Admin connects MetaMask → contract reads `superAdmin()` → role detected → admin dashboard shown.
2. Admin creates election → `createElection()` tx → `ElectionCreated` event emitted → frontend re-queries events → sidebar list updates.
3. Admin adds candidates and authorises voter wallets via individual signed transactions; each write emits a corresponding event.
4. Admin starts election → `ElectionState` transitions `Created` → `Active`.
5. Voter connects → role detected → voter panel shown. Candidate list is fetched by querying `CandidateAdded` events and filtering by `electionId` in JavaScript (required because event parameters are not `indexed` in the deployed contract).
6. Voter selects candidate → `vote()` tx → MetaMask confirmation → `VoteCast` event emitted → `hasVoted` flag set in frontend.
7. Admin ends election → state → `Ended` → `getCandidateVotes()` now accessible → bar chart rendered.

3.3 Component Summary

Table 1: System components and responsibilities

Component	Technology	Role
Smart contract	Solidity 0.8.20	On-chain state, access control, event log
Deploy script	Hardhat + Ethers	One-click Sepolia deployment
Frontend	React + Vite	Admin and voter UI, wallet integration
Wallet	MetaMask	Key management, transaction signing
RPC provider	Alchemy / Sepolia	Ethereum node access (free tier)

4. Smart Contract Design

4.1 State Variables and Data Structures

```

1 enum ElectionState { Created, Active, Ended }
2
3 struct Candidate {
4     uint id;
5     string name;
6     uint voteCount;
7 }
8
```

```

9 struct Election {
10     uint id;
11     string name;
12     ElectionState state;
13     uint candidateCount;
14     mapping(uint => Candidate) candidates;
15     mapping(address => Voter) voters;
16     mapping(address => bool) admins;
17 }
18
19 mapping(uint => Election) private elections;
20 address public superAdmin;
21 uint public electionCount;

```

Listing 1: Core data structures

Elections are stored in a `private` mapping to prevent direct external enumeration; the frontend reconstructs the list from emitted events. Each `Election` contains nested mappings for candidates, voters, and admin addresses, providing $\mathcal{O}(1)$ lookup at the cost of non-iterability.

4.2 Election Lifecycle

The election progresses through three states enforced by `require` guards:

Created $\xrightarrow{\text{startElection()}}$ Active $\xrightarrow{\text{endElection()}}$ Ended

Candidates can only be added in `Created` state. Votes can only be cast in `Active` state. Results can only be read in `Ended` state. This state machine prevents premature result leakage and invalid transitions.

4.3 Access Control

Two-tier role model:

- **Super Admin** (contract deployer): creates elections, assigns election-level admins.
- **Election Admin** (any address the super admin grants): adds candidates, authorizes voters, starts and ends the election.

```

1 modifier onlySuperAdmin() {
2     require(msg.sender == superAdmin, "Not super admin");
3     _;
4 }
5
6 modifier onlyElectionAdmin(uint _electionId) {
7     require(
8         elections[_electionId].admins[msg.sender],
9         "Not election admin"
10    );
11    _;
12 }

```

Listing 2: Role enforcement modifiers

4.4 Vote Integrity

Three on-chain checks prevent vote manipulation:

```

1 require(e.state == ElectionState.Active, "Election not active");
2 require(voter.isAuthorized, "Not authorized");
3 require(!voter.hasVoted, "Already voted");
4 require(_candidateId > 0 && _candidateId <= e.candidateCount,
5         "Invalid candidate");

```

Listing 3: Vote validation logic

The `hasVoted` flag is set atomically within the same transaction as the vote, making double-spending impossible without a separate transaction from the same address—which would be rejected by the second `require`.

4.5 Event Architecture

Seven events are emitted to enable gas-free data retrieval:

Table 2: Contract events and their purpose

Event	Purpose
<code>ElectionCreated(electionId, name)</code>	Populate election list
<code>CandidateAdded(electionId, id, name)</code>	Populate candidate table
<code>VoterAuthorized(electionId, voter)</code>	Display authorized voters
<code>VoteCast(electionId, voter)</code>	Check if wallet already voted
<code>ElectionStarted(electionId)</code>	Detect state change
<code>ElectionEnded(electionId)</code>	Detect state change
<code>AdminAssigned(electionId, admin)</code>	Audit admin delegation

Important deployment note: None of the event parameters are declared `indexed` in the deployed contract. Ethers.js v6 raises an `INVALID_ARGUMENT` error if a topic filter is applied to a non-indexed parameter (`rc.filters.CandidateAdded(electionId)` would fail). The correct pattern is to fetch all events of that type and filter by `electionId` in JavaScript.

5. Frontend Implementation

5.1 Architecture Decisions

No React Router. Previous iterations used React Router for `/admin` and `/vote` routes. This introduced a timing race: when `navigate("/admin")` fired after `setIsAdmin(true)`, the `AdminPage` component rendered before the state update settled, the `isAdmin === false` guard triggered, and the user was immediately redirected away. The solution is a single `view` state variable (`"connect" | "admin" | "voter"`) that is set atomically alongside all other auth state before any conditional render occurs.

No component defined inside App. Defining subcomponents as `const Shell = () =>...` inside `App` causes React to treat each re-render as a new component type, unmounting and remounting the entire subtree. This destroys input focus after every keystroke. All components (`Topbar`, `ElectionSidebar`, `ConfirmModal`, etc.) are defined at module scope.

Stable input handlers. React `useState` setters are referentially stable across renders. Passing them directly as `onChange` props ensures no unnecessary re-renders cascade from form inputs.

5.2 Key Frontend Features

Table 3: Frontend feature inventory

Feature	Description
Role auto-detection	On connect, <code>superAdmin()</code> is read; admin/voter view is set automatically
Auto-Sepolia switch	<code>wallet_switchEthereumChain</code> + <code>wallet_addEthereumChain</code> ensure correct network
Election dashboard	Sidebar lists all elections from <code>ElectionCreated</code> events with state badges and vote counts
Hide ended toggle	Checkbox filters out completed elections from sidebar without redeploying
Stats card	Per-election summary: candidates, authorised voters, votes cast, turnout %, current leader
Confirmation modal	Requires explicit confirmation before ending an election (irreversible action)
Auto-dismiss toasts	Status messages auto-clear after 5 seconds
SVG bar chart	Built-in results visualisation — no external dependency
Tx history log	All confirmed transactions listed with Etherscan links
Copy address button	One-click clipboard copy for voter addresses
Commit-reveal UI	Frontend generates <code>keccak256(candidateId:secret)</code> commitment, prototyping privacy-preserving voting
Polling	Results refresh every 12 seconds during active elections
Reset section	Admin sees exact redeploy command for starting fresh

5.3 Event Querying Strategy

Public Sepolia RPC nodes reject `eth_getLogs` queries spanning more than $\sim 500,000$ blocks. The frontend computes a safe range:

```
1 const latest = await provider.getBlockNumber();
2 const fromBlock = Math.max(0, latest - 500000); // ~70 days
```

All event queries use this range. Since no parameters are indexed, all events of each type are fetched and filtered client-side by `electionId`. This adds minimal overhead for a typical deployment with < 100 elections.

5.4 Commit-Reveal Privacy Prototype

Full on-chain vote privacy requires a two-phase protocol. Phase 1 (commit): the voter submits $H = \text{keccak256}(\text{candidateId} \parallel \text{secret})$ — a hash that reveals nothing about the candidate choice. Phase 2 (reveal): after the election closes, the voter submits the plaintext (`candidateId, secret`); the contract verifies H matches and counts the vote.

The frontend generates the commitment client-side:

```
1 const commitment = ethers.keccak256(
2   ethers.toUtf8Bytes(`${candidateId}:${secret}`)
3 );
```

Full implementation requires adding `commitVote(uint electionId, bytes32 commitment)` and `revealVote(uint electionId, uint candidateId, string secret)` to the contract. The current prototype demonstrates the cryptographic commitment step.

6. Security Analysis

6.1 Threats Addressed

Table 4: Security threat model

Attack	Protection	Mechanism
Vote tampering post-submission	Blockchain immutability	EVM consensus; tx cannot be altered
Double voting	<code>hasVoted</code> flag	Set atomically in same tx as vote
Unauthorised voting	<code>isAuthorized</code> check	Admin whitelist per election
Fake result reporting	On-chain counting	<code>voteCount</code> incremented by contract, not client
Admin privilege escalation	Modifier guards	<code>onlySuperAdmin</code> / <code>onlyElectionAdmin</code>
Front-running vote	Tx ordering	No exploitable economic incentive in this model
Sybil attack (fake wallets)	Off-chain identity	Admin whitelisting is the identity boundary

6.2 Known Limitations

Voter identity. A wallet address is not a verified human identity. The system assumes off-chain identity verification occurs before the admin authorises a wallet. This is the same trust model used in all current blockchain governance systems.

Vote privacy. Public chains are fully transparent. Every vote transaction, including `msg.sender` and `candidateId`, is visible on-chain. The commit-reveal prototype addresses this but requires a contract upgrade.

Arnold Cat period / no formal audit. This system has not been formally audited. Known attack surfaces include: a compromised admin private key granting full election control, and the finite-precision security of `keccak256` commitments being susceptible to dictionary attacks if the secret is weak.

6.3 Gas Analysis

Table 5: Estimated gas costs per operation (Ethereum Sepolia)

Operation	Gas (approx.)	Cost @ 10 Gwei
Deploy contract	~900,000	~0.009 ETH
<code>createElection</code>	~110,000	~0.0011 ETH
<code>addCandidate</code>	~80,000	~0.0008 ETH
<code>authorizeVoter</code>	~45,000	~0.0005 ETH
<code>vote</code>	~55,000	~0.0006 ETH
Read operations	0	Free (view calls)
Event queries	0	Free (off-chain RPC)

All read operations (fetching elections, candidates, voters, results) are view-function calls or event log queries — both gas-free. Only state-mutating transactions incur gas. Voters pay only the `vote()` gas; all administrative gas is borne by the admin wallet.

7. Deployment

7.1 Technology Stack

Table 6: Technology stack

Layer	Technology	Version
Smart contract language	Solidity	0.8.20
Contract framework	Hardhat	2.22.x
Blockchain	Ethereum Sepolia testnet	—
Frontend framework	React + Vite	18.x / 8.x
Blockchain library	Ethers.js	6.16.0
Wallet	MetaMask	browser extension
Node.js	—	v22 (LTS)

7.2 Deployment Process

1. **Environment setup.** Install Node.js and Hardhat; configure `.env` with Alchemy Sepolia RPC URL and deployer private key.
2. **Compilation.** `npx hardhat compile` compiles `Voting.sol` to EVM bytecode and generates the ABI.
3. **Deployment.** `npx hardhat run scripts/deploy.js --network sepolia` sends the deployment transaction; the contract address is printed.
4. **Configuration.** `CONTRACT_ADDRESS` in `contract.js` is updated with the deployed address.
5. **Frontend.** `cd voting-frontend && npm run dev` starts the Vite dev server on `localhost:5173`.

Live contract: <https://sepolia.etherscan.io/address/0xa4659c5Bce9aA467b12C0805c3ab32f03378C67>

8. Results and Demo Flow

A complete end-to-end demo proceeds as follows:

1. **Admin connects** deployer wallet → auto-routed to admin dashboard.
2. **Create election** “IIT Kanpur Student Council 2025”.
3. **Add candidates:** Alice Sharma, Bob Mehta, Carol Singh.
4. **Authorise voter wallets** (one MetaMask signature per wallet).
5. **Start election** → live dot appears in sidebar.
6. **Switch to voter view** (or use a different wallet) → candidates are visible; voter identity card shows the authorised address.
7. **Cast vote** → MetaMask prompts for signature → vote recorded on Ethereum Sepolia → “Already voted” state set.
8. **Admin ends election** → confirmation modal → state transitions to Ended.
9. **Load results** → SVG bar chart rendered with winner highlighted in gold; results table sorted by votes.

All intermediate steps are verifiable on Sepolia Etherscan by inspecting the transaction history of the contract address.

9. Discussion

9.1 What Blockchain Solves Here

The core contribution of this architecture over a conventional database-backed voting system is the **trust model**. In a traditional system, a server administrator can modify vote counts without detection. Here, the counting logic is encoded in the contract bytecode verified against the on-chain hash, executed identically by all EVM nodes, and every mutation is permanently logged. An auditor needs only to read the chain — no privileged access to a server is required.

9.2 Comparison with EVM-based Electronic Voting

India's Electronic Voting Machines record votes on a physical VVPAT paper trail and a sealed memory unit. The machine itself is the trusted authority. VoteChain replaces this single trusted device with a distributed trust model: no single party can alter results. However, EVM machines operate without internet connectivity, providing physical tamper resistance that on-chain systems cannot match. The optimal architecture for national-scale elections likely combines both: physical isolation for the act of voting and blockchain for the audit trail.

10. Future Work

- **Full commit-reveal voting.** Add `commitVote()` and `revealVote()` to the contract to enforce vote confidentiality during the voting window.
- **ZK-proof integration.** Zero-knowledge proofs (e.g., zk-SNARK-based ballot schemes) can prove a voter is authorised without revealing which candidate they chose, achieving both authentication and privacy.
- **Indexed events.** Redeploy with `indexed` on `electionId` in all events to enable efficient on-chain topic filtering and reduce client-side processing.
- **IPFS metadata.** Store candidate photos, manifestos, and election descriptions on IPFS; reference the content hash on-chain.
- **NIST SP 800-56A compliant key management.** Replace MetaMask private key management with hardware wallet integration (Ledger/Trezor) for high-security deployments.
- **Multi-signature admin.** Require k -of- n admin signatures to start or end an election, preventing single-point-of-failure admin compromise.
- **Mainnet deployment with real identity layer.** Integrate with Aadhaar-linked wallet binding or institutional SSO for authenticated voter registration.

11. Conclusion

VoteChain demonstrates that a functional, secure, and auditable voting system can be built on Ethereum with fewer than 200 lines of Solidity and a React frontend. The key design decisions — immutable on-chain state, event-sourced reads, role-based access control, and MetaMask authentication — collectively produce a system where:

- Vote counts cannot be tampered with after submission.

- Double voting is prevented by cryptographic identity.
- Results are publicly verifiable without any trusted intermediary.
- The admin cannot change votes, only the lifecycle of the election.

The project also surfaces honest engineering trade-offs: blockchain guarantees integrity but not identity or privacy, and every on-chain write has a gas cost that creates friction at scale. Future extensions via commit-reveal schemes and ZK proofs address the privacy gap, while institutional identity verification (e.g., Aadhaar binding) addresses the identity gap. Together these make decentralised voting a credible architecture for real-world deployment.

References

- [1] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. <https://bitcoin.org/bitcoin.pdf>
- [2] V. Buterin, “Ethereum: A next-generation smart contract and decentralised application platform,” 2014. <https://ethereum.org/en/whitepaper/>
- [3] Solidity Documentation, *Solidity v0.8.20*, Ethereum Foundation, 2024. <https://docs.soliditylang.org>
- [4] R. Rimple, *Ethers.js Documentation v6*, 2024. <https://docs.ethers.org/v6/>
- [5] R. Cramer, R. Gennaro, and B. Schoenmakers, “A secure and optimally efficient multi-authority election scheme,” *Advances in Cryptology (EUROCRYPT ’97)*, Springer LNCS, 1997.
- [6] J. Benaloh and D. Tuinstra, “Receipt-free secret-ballot elections,” *Proc. 26th ACM STOC*, 1994, pp. 544–553.
- [7] Nomic Foundation, *Hardhat Documentation*, 2024. <https://hardhat.org>